

Project Report

Deep Learning and Generative-AI

Smart MCQ Solver Challenge 2026, Kaggle
Conducted by: IIT Madras

Naini Diwan

July 26, 2026

Contents

1	Introduction	3
2	Basic Concepts	3
2.1	TF-IDF and Cosine Similarity	3
2.2	Transformers	3
2.3	Parameter-Efficient Fine-Tuning with LoRA	4
2.4	Model Ensembling	4
2.5	Retrieval-Augmented Generation (RAG)	4
2.6	Evaluation Metrics	5
3	Methodology	5
3.1	Dataset	5
3.2	Data Preprocessing	5
3.3	Classification Track: Four Models	6
3.4	Retrieval-Augmented Generation	7
4	Results	8
4.1	Observations	8
5	Discussion	9
5.1	Fine-Tuning vs Retrieval-Augmented Zero-Shot Prompting	9
5.2	Why the RAG Pipeline Underperformed	9
5.3	Why DeBERTa Outperformed LoRA-Adapted RoBERTa	9
5.4	Value of the Ensemble	9
5.5	Limitations	10
6	Future Work	10
7	References	11

1 Introduction

Multiple-choice question answering (MCQA) is a benchmark task in NLP – Natural Language Processing. Given a question (the *prompt*) and a small set of candidate answers, a system must identify the correct option, or in the variant considered here, produce its best three guesses in order of confidence. MCQA underlies automated tutoring systems, exam-grading assistants, trivia and quiz engines, and is a standard proxy task for measuring how well a model has internalised factual and reasoning knowledge from its training data or from an external source.

This report documents my work on the *Smart MCQ Solver Challenge* dataset. All the models built are evaluated using Accuracy, Macro F1, and Mean Average Precision at 3 (MAP@3). Fine-tuned encoder ensemble achieved the strongest performance (MAP@3 = 0.9986), substantially ahead of the TF-IDF baseline (MAP@3 = 0.2973). The training data spans a broad mixture of STEM and humanities questions, making the task a useful stress test for both closed-book (parametric) and open-book (retrieval-augmented) question-answering strategies.

Two kaggle notebooks were used to explore this problem:

1. A **fine-tuning notebook** that trains four systems of increasing sophistication: (1) unsupervised TF-IDF + cosine-similarity baseline, (2) fully fine-tuned **DeBERTa-v3-small** classifier, (3) **RoBERTa-base** classifier adapted with Low-Rank Adaptation (LoRA), and (4) a probability-weighted ensemble of the two transformer classifiers.
2. A **retrieval-augmented generation notebook** that embeds a sample of STEM Wikipedia passages, indexes them with FAISS for nearest-neighbour retrieval, and prompts an open-weight instruction-tuned language model (**Phi-3-mini-4k-instruct**) to rank the answer options using the retrieved context, with no task-specific fine-tuning.

The goal of this report is threefold: (1) to explain the conceptual building blocks behind each approach, (2) to document the data-processing and modelling workflow, and (3) to compare the resulting metrics of evaluation, discussing why the different strategies perform as they do and what could be improved in future iterations.

2 Basic Concepts

2.1 TF-IDF and Cosine Similarity

Term Frequency–Inverse Document Frequency (TF-IDF) [11] weights each word in a document by how often it appears in the corpus, so that rare terms are emphasised over generic ones. The question prompt and each candidate option could be represented as TF-IDF vectors in a shared vocabulary space. This allows their semantic overlap to be approximated with **cosine similarity**, which measures the angle between two vectors independently of their length. This provides a fast, unsupervised baseline for MCQA: the option whose vector is closest in direction to the prompt’s vector is predicted as most likely correct.

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

2.2 Transformers

The transformer is a family of artificial neural network architectures based on the multi-head attention mechanism, in which text is converted to numerical representations called tokens, and each token is converted into a vector via lookup from a word embedding table. Transformer encoder models such as BERT [?], RoBERTa [2], and DeBERTa [3] are pretrained on large text corpora using masked-language-modelling objectives. They can be adapted to

multiple-choice classification by concatenating the prompt with each candidate option, encoding (question, option) pairs, and passing the pooled representation of each pair through a shared linear classification head to obtain one logit per option.

RoBERTa [2]

The *Robustly Optimized BERT Pretraining Approach* retains the same transformer encoder architecture as BERT. Its pretraining objectives include dynamic masking, sentence packing, larger batches and a byte-level BPE tokenizer.

DeBERTa [3]

It improves on BERT/RoBERTa with a *disentangled attention* mechanism that represents each token with two separate vectors, one for content and one for relative position. It computes the attention weights from both content-to-content and content-to-position interactions, together with a mask decoder that reincorporates absolute position information before the final prediction layer. This gives the model a richer understanding of word order and relationships as compared to models that fuse position and content into a single embedding.

2.3 Parameter-Efficient Fine-Tuning with LoRA

Fully fine-tuning a large pretrained model updates every parameter and can be memory and compute-intensive. **Low-Rank Adaptation (LoRA)** [4] instead freezes the original weight matrices and injects trainable low-rank *adapter* matrices into selected layers (commonly the attention query and value projections).

$W_0 + \Delta W = W_0 + BA$, where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ and $\text{rank } r \ll \min(d, k)$

Only A and B are updated during training, which appreciably reduces the number of trainable parameters while retaining performance close to full fine-tuning [5].

2.4 Model Ensembling

An **ensemble** combines predictions of multiple models to reduce variance and exploit complementary strengths. In this study, the probability distributions from two independently trained classifiers are combined via a fixed weighted average, before re-ranking the five options.

$$p_{\text{ens}} = w_1 p_1 + w_2 p_2$$

2.5 Retrieval-Augmented Generation (RAG)

RAG [7] couples a generative language model with a non-parametric retrieval step over an external corpus, allowing the model to ground its answer in retrieved evidence rather than relying purely on knowledge memorised during pretraining. The pipeline requires three components:

- **Dense Text Embedder**, here `all-MiniLM-L6-v2`, a Sentence-BERT-style model [6] that maps text to a vector such that semantically similar texts have nearby embeddings.
- **Vector Index** for efficient nearest-neighbour search over the embedded corpus. This study uses **FAISS** with an `IndexFlatIP` (inner-product) index. Since the embeddings are L2-normalised, inner product is equivalent to cosine similarity.
- **Generative Language Model** that consumes the retrieved passages together with the question and produces an answer. This study uses **Phi-3-mini-4k-instruct** [8], a 3.8-billion-parameter instruction-tuned model designed to run efficiently on constrained hardware, prompted in a zero-shot fashion to output a ranked list of option letters.

2.6 Evaluation Metrics

Three metrics are used throughout this study:

- **Accuracy** – the fraction of examples for which the top-ranked prediction matches the true label.
- **Macro F1** – the unweighted mean of the per-class F1 scores, giving equal importance to each of the five answer options regardless of class frequency.
- **MAP@3** (Mean Average Precision at 3) – In our case of exactly one correct label per query, N being the total number of questions in the set, MAP@3 is defined as

$$\text{MAP@3} = \frac{1}{N} \sum_{i=1}^N [\text{AP@3}]_i$$

where,

$$\text{AP@3} = \begin{cases} \frac{1}{k} & , \text{ if the correct option is at rank } k = 1, 2, \text{ or } 3 \\ 0 & , \text{ otherwise} \end{cases}$$

3 Methodology

3.1 Dataset

The *Smart MCQ Solver Challenge* dataset consists of a labelled training set (`train.csv`) and an unlabelled test set (`test.csv`), each with the columns `id`, `prompt`, `A`, `B`, `C`, `D`, `E`, and (only for training data) `answer`. The raw training file contains 2,000 rows with no missing values in any of the question/option columns. For the RAG pipeline, a subsample of a pre-processed STEM Wikipedia corpus [12] (5,000 passages) is used as the retrieval knowledge base.

3.2 Data Preprocessing

Before training, the data is processed in six steps, illustrated in Figure 1:

- (1) a null-value check confirms there are no missing entries;
- (2) rows with duplicate question prompts are identified (242 duplicates found) and all but the first occurrence are dropped, reducing the dataset from 2,000 to 1,758 unique rows;
- (3) case normalisation;
- (4) common instructional boiler-plate phrases (“pick the best possible answer:”, “select the most accurate option:”, etc.) are removed (whenever they exist in the question) via regular expressions, so that the model focuses on the substantive question text;
- (5) the categorical `answer` column is encoded using `scikit-learn`’s `LabelEncoder` [10]; and
- (6) the cleaned data is split into training and validation subsets using a **stratified** split on the label, which guarantees that both subsets preserve the same class balance across options A–E, yielding 1,406 training and 352 validation examples.

Token-length analysis on cleaned prompts showed a 99th-percentile length of 44 tokens, justifying the fixed tokenisation length of 128 tokens for the transformer models. RAG pipeline (Section 3.4) operates on `train.csv` and `test.csv` files without this preprocessing stage.

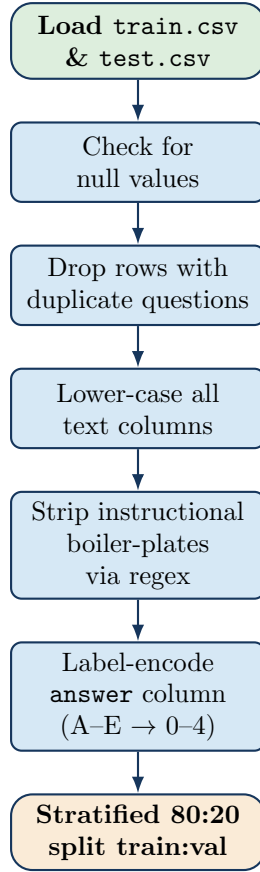


Figure 1: Data preprocessing

3.3 Classification Track: Four Models

Figure 2 shows the four systems built and evaluated on the preprocessed data.

1. **TF-IDF baseline** – `TfidfVectorizer` (with `max_df=0.95` to avoid highly occurring words) is fit jointly over prompt and option columns so that they share same vector space. For each validation example, cosine similarity between prompt vector and each option vector is computed, and 5 similarities are ranked directly as the prediction scores (no training here).
2. **Fine-tuned DeBERTa** – The `microsoft/deberta-v3-small` checkpoint is loaded with a multiple-choice head (`AutoModelForMultipleChoice`) and fully fine-tuned for 4 epochs with a cosine learning-rate schedule (peak learning rate 2×10^{-6} , 10% warm-up, gradient clipping at 0.5, effective batch size 32 via 2-step gradient accumulation).
3. **LoRA-adapted RoBERTa** – The `roberta-base` checkpoint is wrapped with a LoRA adapter (rank $r = 8$, $\alpha = 16$, dropout 0.1) applied to the query and value projection matrices. Then it is trained for 12 epochs at a learning rate of 5×10^{-4} ; a markedly higher rate than full fine-tuning, since only the small adapter matrices are updated.
4. **Weighted Ensemble** – The softmax probabilities of DeBERTa and LoRA-RoBERTa models are combined as $P_{\text{ens}} = 0.70 P_{\text{DeBERTa}} + 0.30 P_{\text{RoBERTa}}$, favouring the stronger model and the combined probabilities are re-ranked to produce the final top-3 predictions.

The transformer models share a common `HuggingFaceMCQDataset` class that tokenises the prompt paired with each of the five options into fixed-length [5, 128] tensors, and a data collator that stacks these into tensors of shape [batch, 5, 128] for the `Trainer` API.

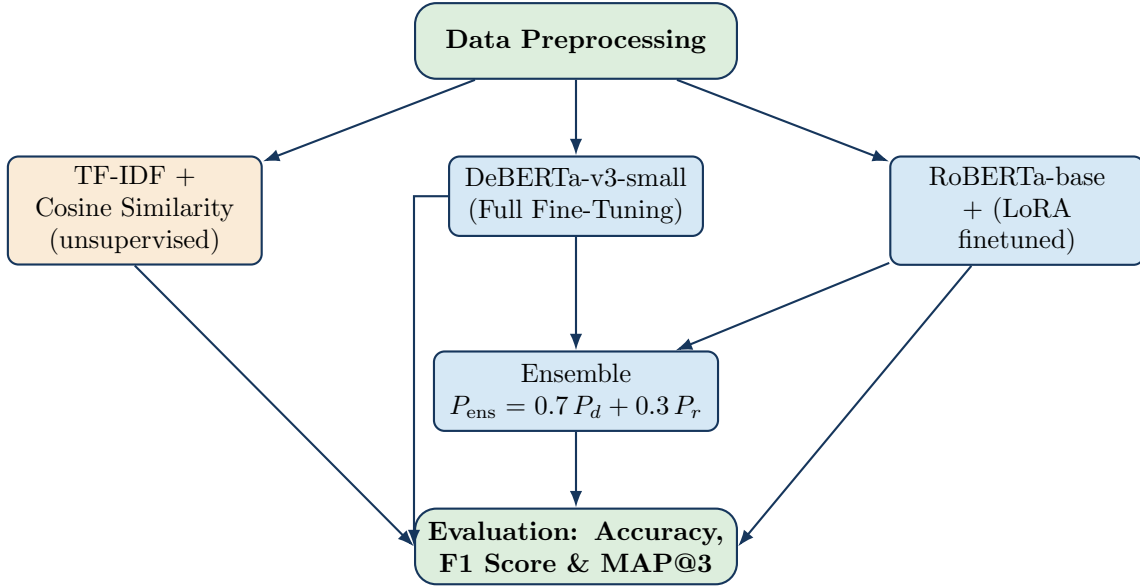


Figure 2: Classification (fine-tuning) track: four systems evaluated on the same validation split.

3.4 Retrieval-Augmented Generation

A random sample of 5,000 passages is drawn from the STEM Wikipedia corpus and embedded with the `all-MiniLM-L6-v2` sentence embedder (384-D, L2-normalised vectors), then indexed with a FAISS `IndexFlatIP` index for inner-product nearest-neighbour search. At inference, each question prompt is embedded with the same encoder and used to retrieve its 3 nearest passages. These passages are concatenated into a context block and inserted, together with the question and options, into a fixed prompt template. This instructs `Phi-3-mini-4k-instruct` to respond with only 3 most likely option letters in order. Generation is run greedily (`do_sample=False`) with a short output budget (10 new tokens). Generated text is parsed by scanning for the first occurrence of each valid option letter. Neither retriever nor the language model are finetuned.

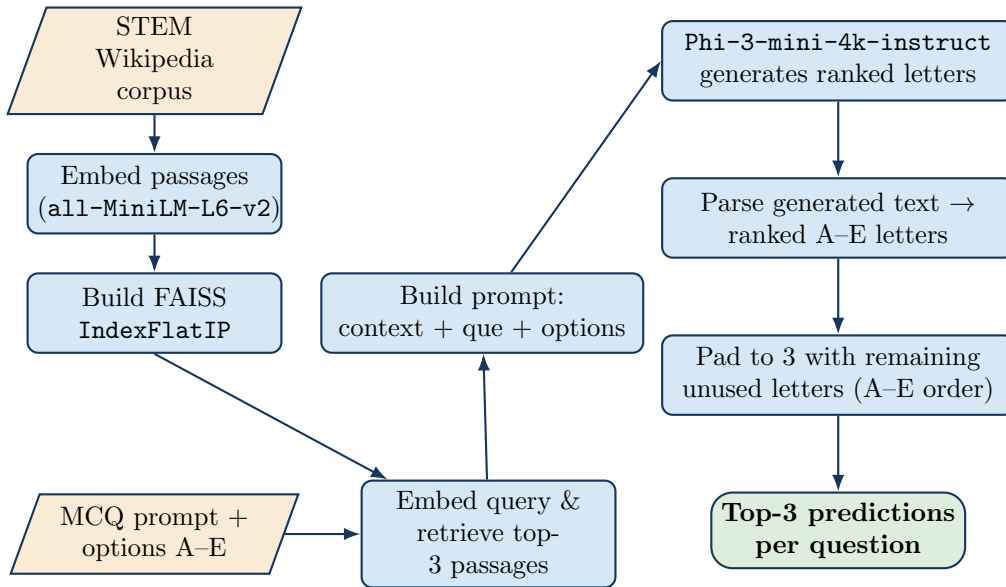


Figure 3: RAG: FAISS-based passage retrieval feeding a Phi-3-mini generator

4 Results

Classification models are evaluated on their respective validation splits. The RAG pipeline is evaluated on a separate random sample of 200 examples drawn from the raw training file.

Table 1: Validation performance of all five systems.

Model	Accuracy	Macro F1	MAP@3
TF-IDF + Cosine Similarity	0.1761	0.1715	0.2973
DeBERTa-v3-small (full fine-tune)	0.9972	0.9972	0.9986
RoBERTa-base + LoRA	0.9631	0.9609	0.9778
Weighted Ensemble (0.70:0.30)	0.9972	0.9970	0.9986
RAG: FAISS + Phi-3-mini-4k-instruct	0.6100	0.5996	0.7625

Figure 4 visualises the results as a bar chart, making the gap between the fine-tuned transformer systems, the zero-shot RAG pipeline, and the TF-IDF baseline immediately apparent.

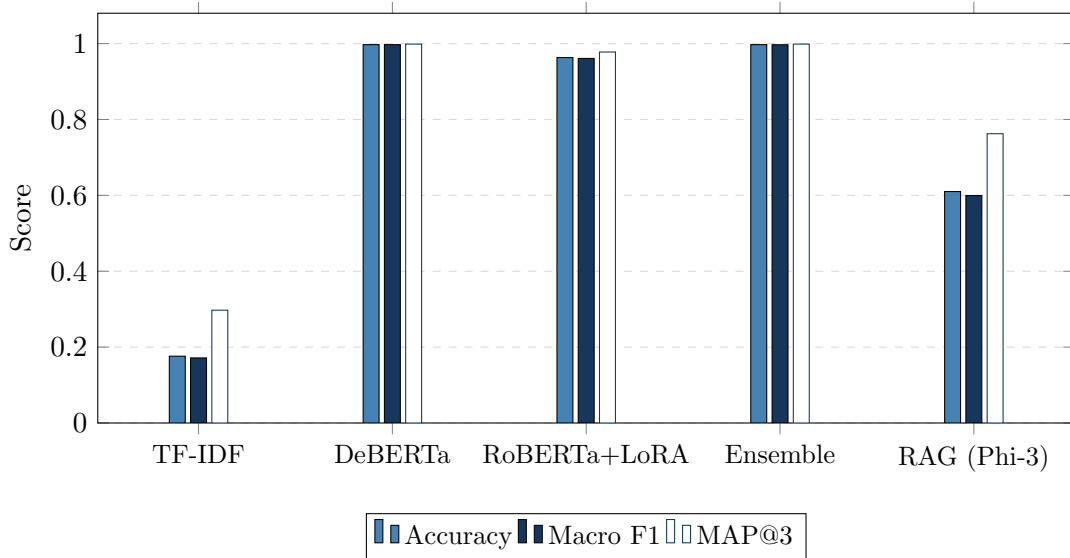


Figure 4: Inference Results

4.1 Observations

- **TF-IDF baseline** performs only modestly above the random-guessing floor for a five-option question (0.20 accuracy at chance), confirming that pure lexical overlap between a question and its options is a weak signal for this dataset – correct and incorrect options are often lexically similar (e.g. paraphrased variants of the same statement), so cosine similarity on sparse bag-of-words vectors cannot reliably separate them.
- **DeBERTa-v3-small**, despite being the smallest of the two transformer checkpoints, achieved the best individual-model scores after only 4 epochs of full fine-tuning, converging to a near-perfect MAP@3.
- **RoBERTa-base + LoRA** required more epochs and a much higher learning rate to reach a lower, though still strong, MAP@3 of 0.9778.
- **Ensemble** matched DeBERTa’s already-high scores, showing that a 70:30 weighting preserves stronger model’s performance while still incorporating other model’s signal.

- **RAG** outperformed the TF-IDF baseline (MAP@3 0.76 vs. 0.30) purely through zero-shot prompting and retrieval, but fell well short of the fine-tuned transformer classifiers.

5 Discussion

5.1 Fine-Tuning vs Retrieval-Augmented Zero-Shot Prompting

When labelled training data is available, direct supervision on the target task and label space is usually more sample-efficient than relying on a general-purpose model’s zero-shot reasoning, even when that model is grounded with retrieved evidence. The classification models directly optimise the same metric family (accuracy/F1/MAP@3-adjacent cross-entropy loss) that is used for evaluation. Also, their architecture (five parallel prompt-option encodings feeding the same classification head) is purpose-built for the problem statement, whereas the RAG pipeline must implicitly perform the comparison inside free-form text generation and then have that generation parsed back into a ranking.

5.2 Why the RAG Pipeline Underperformed

Structural factors likely limit the RAG pipeline’s ceiling relative to the fine-tuned encoders:

- **Domain mismatch and corpus coverage.** The Wikipedia knowledge base is restricted to a *STEM* subset, yet the question set includes non-STEM material (e.g. Heideggerian philosophy); moreover, only a 5,000-passage random subsample of the full corpus is indexed (chosen for speed within Kaggle session limits). Thus, many questions may have no closely relevant passage retrievable at all.
- **Small, non-fine-tuned generator.** Phi-3-mini is used zero-shot with a single prompt template and a short 10-token generation budget; it never sees any labelled training example for this task, unlike the encoder models, which see 1,406 labelled examples across multiple epochs.
- **Answer-extraction brittleness.** The ranking is recovered by scanning generated text for option letters, with any missing letters padded in a fixed fallback order (A, B, C, D, E). This heuristic can inject essentially arbitrary rank-3 guesses whenever the model does not clearly output three distinct letters, which depresses MAP@3.
- **Evaluation-set mismatch.** The RAG pipeline is evaluated on a smaller sample of the *original, non-duplicated* training file, whereas the classification models are evaluated on a stratified split of the cleaned 1,758-row dataset. Notwithstanding that their evaluation results are informative, they may not be perfectly comparable like-for-like.

5.3 Why DeBERTa Outperformed LoRA-Adapted RoBERTa

DeBERTa’s disentangled content/position attention and enhanced mask decoder give it a stronger inductive bias for fine-grained textual comparison tasks such as distinguishing subtly different answer options, and full fine-tuning allows every layer to adapt to the target task. RoBERTa with LoRA updates only a small number of low-rank adapter parameters in the attention query/value projections, which is far more parameter- and memory-efficient but, as observed here, needed three times as many training epochs and a $250\times$ higher learning rate to approach (without quite reaching) DeBERTa’s performance.

5.4 Value of the Ensemble

The ensemble’s benefit is modest in this study because one of the two constituent models (DeBERTa) was already close to a performance ceiling (MAP@3 > 0.998) on this validation

split; ensembling tends to add the most value when the constituent models make more varied and less correlated errors, or when no single model is already near-saturated.

5.5 Limitations

- All classification results come from a single train/validation split and a single training run per model; no cross-validation or repeated-seed variance estimate is reported, so small score differences should be treated with caution.
- The ensemble weights were fixed without being tuned via a systematic search (e.g. grid search or a small held-out validation sweep).
- The RAG pipeline indexes only a 5,000-passage random subsample of the available Wikipedia corpus, and RAG evaluation-set's size differs from the classification models'. Hence the reported numbers are directionally, but not strictly, comparable.
- Generation length for RAG model is capped at 10 tokens, and answer extraction relies on a simple parsing heuristic with a fixed fallback order, which can mask the model's true ranking ability.

6 Future Work

Building on the findings above, the following directions could improve and extend this study:

1. **Systematic hyperparameter & ensemble-weight tuning.** Replace fixed ensemble weighting with a validated grid or Bayesian search, and similarly tune the hyperparameters of RoBERTa model and DeBERTa.
2. **Larger corpus retrieval for RAG.** Index entire STEM Wikipedia dump rather than a subsample, and experiment with a larger k and re-ranking of retrieved passages.
3. **Fine-tuning the RAG generator.** Instead of zero-shot prompting, fine-tune Phi-3-mini (or any suitable instruction-tuned model) on the labelled training questions – potentially with LoRA for efficiency. Increase the generation token budget with a more robust parsing/constrained-decoding scheme instead of the current fallback heuristic.
4. **Larger and alternative encoder backbones.** Evaluate larger checkpoints such as deberta-v3-large or deberta-v3-base, or alternative architectures, to test whether it generalises to a harder or more diverse question set.
5. **Cross-validation and robustness testing.** Repeat training for multiple random seeds and k-fold splits to obtain confidence intervals on Accuracy, Macro F1, and MAP@3.
6. **Hybrid fusion of both tracks.** Combine the calibrated probabilities from the fine-tuned encoder ensemble with a retrieval-grounded signal from the RAG pipeline (e.g. as an additional ensemble member or a re-ranking feature), to test whether external knowledge grounding can still add value on top of an already strong supervised classifier, particularly for *out-of-distribution* or knowledge-intensive questions.
7. **Deployment as an interactive tool.** Package the best-performing pipeline behind a lightweight interface (e.g. a Gradio or Streamlit application) to allow interactive exam-assistance or self-study use cases, including displaying the model's confidence and the retrieved evidence (for the RAG variant) to support user trust and error inspection.

7 References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is All you Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [3] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa: Decoding-enhanced BERT with Disentangled Attention,” in *Proc. International Conference on Learning Representations (ICLR)*, 2021. arXiv:2006.03654.
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [5] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan, “PEFT: State-of-the-art Parameter-Efficient Fine-Tuning Methods,” Hugging Face, GitHub repository, 2022.
- [6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019. arXiv:1908.10084.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.11401.
- [8] M. Abidin *et al.*, “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone,” Microsoft, *arXiv preprint arXiv:2404.14219*, 2024.
- [9] T. Wolf *et al.*, “Transformers: State-of-the-Art Natural Language Processing,” in *Proc. EMNLP: System Demonstrations*, 2020.
- [10] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [11] K. Spärck Jones, “A Statistical Interpretation of Term Specificity and Its Application in Retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [12] M. Banaei, “STEM Wikipedia (Cohere, No Embeddings)” and “All Paraphs Parsed Expanded” (Kaggle datasets), <https://www.kaggle.com/datasets/mbanaei>, accessed 2026.